CS 2001 – Data Structures
Assignment no. 1



NATIONAL UNIVERSITY
OF COMPUTER & EMERGING SCIENCES
MULTAN CAMPUS

FAST

Inctructor : Dr Usman Joyia

Submitted by : Ibraheem chand
Roll No. : 25M- 3067
Section: BSSE-3A

Task No 1:

```
#include <iostream>

using namespace std;


int main()

{

   int N ;

   int M ;


   cout << "Enter number of students : ";

   cin >> N;

   cin.ignore();

   cout << "Enter number of assessments : ";

   cin >> M;

   cin.ignore();


   int** marks = new int*[N];


   for (int i = 0; i < N; i++)

   {

      marks[i] = new int[M];

   }




   for (int i = 0; i < N; i++)

   {

      for (int j = 0; j < M; j++)

      {

         cout << "Enter marks for Student " << i + 1 << ", Assessment " << j + 1 << " ";
```

```
        cin >> marks[i][j];
    }
  }

  for (int i = 0; i < N; i++)
  {
    int total = 0;

    for (int j = 0; j < M; j++)
    {
      total += marks[i][j];
    }

    cout << "Total marks of Student no. " << i + 1 << "   =    " << total << endl;
  }

  int diagonalSum = 0;        //A

  /*
  The sum of diagonal elements will run till the smallest number in between M and N, n = min(M,N)  , thus BigO Notation becomes O(n)
  */
  for (int i = 0; i < N && i < M; i++)
  {
    diagonalSum += marks[i][i];
  }
```

```
 int aboveDiagonalSum = 0;        //B

   /*

To find the sum of elements above the diagonal outer loop runs N times while inner loop runs M times , which results in the time Complexity and Big O notation O (N*M)

*/


   for (int i = 0; i < N; i++)

   {

      for (int j = i + 1; j < M; j++)

      {

         aboveDiagonalSum += marks[i][j];

      }

   }


int belowDiagonalSum = 0;        //C


/*

To find the sum of elements Below the diagonal outer loop runs N times while inner loop runs M times , which results in the time Complexity and Big O notation O (N*M)

*/

   for (int i = 1; i < N; i++)

   {

      for (int j = 0; j<i  && j < M; j++)

      {

         belowDiagonalSum += marks[i][j];

      }

   }


   cout << "main diagonal sum = " <<  diagonalSum  << endl;
```

```
    cout << "above main diagonal sum = "  << aboveDiagonalSum << endl;
    cout << "below main diagonal sum = " <<  belowDiagonalSum << endl;


    for (int i = 0; i < N; i++)
    {
        delete[] marks[i];
    }

    delete[] marks;

    return 0;
}
```

Output:

```
PS E:\Languages\DataStructures> cd "e:\Languages\DataStructures\Sept\1st\3rd sept\" ; if ($?) { g++ task1.cpp -o task1 } ; if ($?) { .\task1 }
Enter number of students : 2
Enter number of assessments : 2
Enter marks for Student 1, Assessment 1 5
Enter marks for Student 1, Assessment 2 45
Enter marks for Student 2, Assessment 1 45
Enter marks for Student 2, Assessment 2 47
Total marks of Student no. 1   =     50
Total marks of Student no. 2   =     92
main diagonal sum = 52
above main diagonal sum = 45
below main diagonal sum = 45
PS E:\Languages\DataStructures\Sept\1st\3rd sept>
```

Task No.2

Task NO.2

Expression	Dominant Term	Big O Notation $T(n) = O(?)$
i) $T(n) = \frac{1}{2}n^2 + 1000 \cdot n + 2$	$\frac{1}{2}n^2$	$O(n^2)$
ii) $T(n) = 0.001n^2 + 1000n + 3$	$0.001n^2$	$O(n^2)$
3) $T(n) = 0.001\log_{20} n + \log_{10}\log_{10} n$	$0.001\log_{20} n$	$O(\log n)$
4) $T(n) = 0.1n^{1.2} + 100n\log_{10} n + 1000n$	$0.1n^{1.2}$	$O(n^{1.2})$
5) $T(n) = 0.05n\log_2 n + n(\log_2 n)^2$	$0.05n(\log_2 n)^2$	$O(n\log_2^2 n)$

Task no. 3

Task 3: Extract T(n) from Codes.

Code NO. 1:-

```
for(i=n ; i>0 ; i++){
    for (j=i ; j>=1 ; j--)
        count++;}
```

T(n) is the Sum of (n) and (n) for every i value inner Loop exactly run i times

$$T(n) = \frac{n(n+1)}{2} = \frac{n^2+n}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

Upper bound becomes $= O(n^2)$ ⇒ Takes $C = 1/2$ and $n_0 = 1$

$$T(n) = O(n^2)$$

$$C_1 n^2 \leq T(n) \leq C_2 n^2 \quad \forall \quad n \geq 1$$

Code 3:-

```
for (i=1 ; i<n ; i = i*2)
    for( j=1 ; j<=i ; j++){
        count++; }
```

For each iteration value of i increases by 2^1 while inner loop runs i times Each iteration. ($K \approx \log n$) in K iterations.

$$T(n) = 2^0 + 2^1 + 2^2 + 2^3 \ldots\ldots 2^{(K+1)}$$

We are Calculating upper bound here

$$T(n) = 2^K - 1 \approx n - 1$$

$$T(n) = O(n) \quad \text{Takes } C_1 = 1/2 \text{ and } n_0 = 2$$

Code NO.5

```
for (i=1; i<=n; i=i*2)
    for (j=1; j<=n; j=j*2)
        count ++;
```

Both i and j doubles from 1 to n running $\log_n$ times and both are independent of each other.

$T(n) = \log n \cdot \log n$

$T(n) = O(\log n \cdot \log n)$

$T(n) = O(\log^2 n) \Rightarrow c_1 = c_2 = 1$ and $n_0 = 2$.

Code 6

```
for (i=1; i<=n; i++)
    { for(j=1; j<=i; j=j*2)
        { count++; }.
```

For fixed i in each iteration j doubles itself in each inner iteration showing $\log n$ behaviour.

Combining thus

$T(n) = n \cdot \log n$ Takes $c = 1/21$, $n_0 = 4$.

$T(n) = O(n \log n)$

Code: 2

```
for(i=1 ; i<=n ; i++)
    { for(j=n ; j>1 ; j/=2)
        { count++; }
```

for each iteration of from n to 1 its complexity is nlogn steps while outer Loop take n steps.

$T(n) = n * \log n.$

$T(n) = n \log n$

Takes $c_1 = 1/2$ and $n = 2$.

Code: 4

```
if(n%.2 == 0){
        for(i=0; i<n ; i++)
            count ++;

Else { for(int i=0, i<n ; i++)
        { for (int j=0, j<n, j++)
            { count++; }
```

Best Case
n = Even

$T(n) = n \rightarrow O(n)$ $c_1 = c_2 = 1$ and $n_0 = 1$

Worst Case

$T(n) = n^2 \rightarrow O(n^2)$ $c_1 = c_2 = 1$ and $n_0 = 1$

Task no. 4:

```
#include <iostream>

using namespace std;


class DynamicArry

{

public:




  int* data;

  int size;

  int capacity;


  DynamicArry(int capac){

    if (capac > 0)

    this->capacity = capac;

    else

    this->capacity = 1;

    data= new int[capacity];

    size = 0;

  }

   DynamicArry(DynamicArry& other) {

    capacity = other.capacity;

    size = other.size;

    data = new int[capacity];

    for (int i = 0; i < size; i++) {

      data[i] = other.data[i];

    }

  }


  ~DynamicArry(){
```

```
    delete[] data;
}

 void resize() {

   int newCapacity;

    if (capacity == 0)
    newCapacity = 1;

    else
    newCapacity = capacity * 2;

    int* newData = new int[newCapacity];
    for (int i = 0; i < size; i++) {
       newData[i] = data[i];
    }
    delete[] data;
    data = newData;
    capacity = newCapacity;
}

int search(int value)  {
    for (int i = 0; i < size; i++) {
       if (data[i] == value)
       {
          return i;
       }
    }
    return -1;
}
```

```
  void insert(int value) {
    if (size == capacity) {
      resize();
    }
    data[size] = value;
    size++;
  }

  bool remove(int index) {
    if (index < 0 || index >= size) {
      return false;
    }
    for (int i = index; i < size - 1; i++) {
      data[i] = data[i + 1];
    }
    size--;
    return true;
  }
   void display(){
    cout << "[ ";
    for (int i = 0; i < size; i++) {
      cout << data[i] << " ";
    }
    cout << "] \n size of array = " << size << "\n And the capacity = " << capacity <<  endl;
  }
};

int main(){

  DynamicArry arr(3);
```

```
    arr.insert(91);
    arr.insert(92);
    arr.insert(93);
    arr.insert(94);
    arr.insert(95);

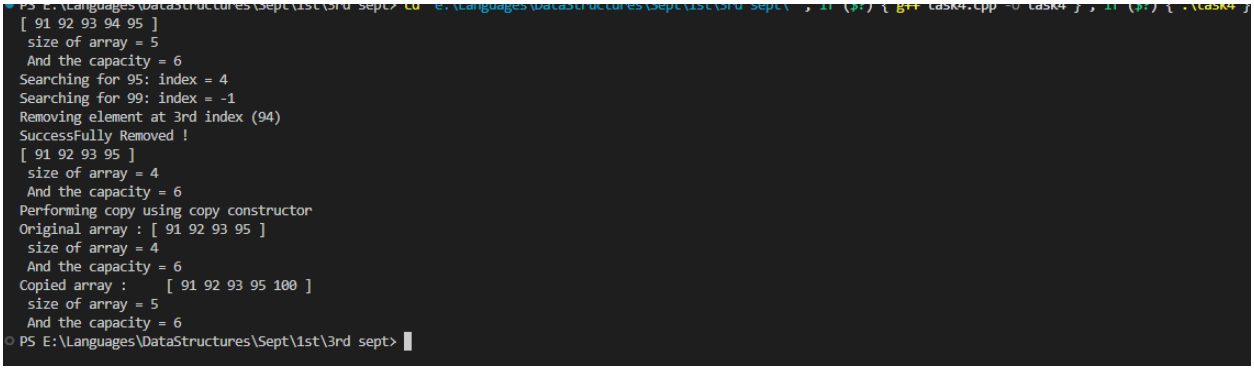
    arr.display();

    cout << "Searching for 95: index = " << arr.search(95) << endl;
    cout << "Searching for 99: index = " << arr.search(99) << endl;

    cout << "Removing element at 3rd index (94) " << endl;
    bool ok = arr.remove(3);
    if (ok)
    {
       cout << "SuccessFully Removed ! " << endl;
    }
     arr.display();

    cout << "Performing copy using copy constructor" << endl;
    DynamicArry copyArr(arr);
    copyArr.insert(100);
    cout << "Original array : ";
    arr.display();
    cout << "Copied array :    ";
    copyArr.display();
}
```

Output:



----------The End ----------